A MINI PROJECT REPORT ON

FACULTY APPOINTMENT BOOKING SYSTEM

A MERN Stack Web Application

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

K.Akshaya - 25MVCSMR0048

L.Akshaya - 25MVCSMR0051

M.Nandini - 25MVCSMR0057

P.Gangothri- 25MVCSMR0069

Department of Computer Science and Engineering

Mallareddy Technical campus

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, [Guide Name], Department of Computer Science and Engineering, for the valuable guidance, constant encouragement and constructive suggestions given throughout the course of this project.

We are thankful to [HOD Name], Head of the Department, and to the Principal of [College Name] for providing the facilities and a supportive environment to carry out this work. We also thank all the faculty members and the non-teaching staff of the department for their help.

Finally, we thank our parents and friends for their continuous support and motivation, and the open-source community whose documentation and tools made this project possible.

TABLE OF CONTENTS

CERTIFICATE 1

ACKNOWLEDGEMENT 2

ABSTRACT 4

CHAPTER 1: INTRODUCTION 5

CHAPTER 2: PROBLEM STATEMENT 6

CHAPTER 3: OBJECTIVES AND SCOPE 7

CHAPTER 4: FEATURES AND FUNCTIONAL REQUIREMENTS 9

CHAPTER 5: ALGORITHMS AND METHODOLOGY 12

CHAPTER 6: TECHNOLOGIES AND IMPLEMENTATION 17

CHAPTER 7: RESULTS, TESTING AND PERFORMANCE 24

CHAPTER 8: CONCLUSION AND FUTURE SCOPE 28

REFERENCES 29

ABSTRACT

Meeting a faculty member for academic guidance, doubt clearing, project discussion or mentoring is an everyday need of every student. In most institutions this process is still handled informally: students wait outside cabins, send repeated messages, or rely on verbal commitments that are easily forgotten. The result is wasted time for students, unpredictable interruptions for faculty and no record of who met whom, when and why.

This project presents the design and development of a Faculty Appointment Booking System built on the MERN stack, that is, MongoDB, Express.js, React.js and Node.js. The system lets faculty members publish the time slots in which they are available, and lets students browse those slots and book one by stating the purpose of the meeting. Every booking is stored in MongoDB together with the student, faculty, date, time and purpose, and moves through a clear life cycle of Pending, Approved, Rejected, Cancelled and Completed.

The backend exposes a set of REST APIs implemented with Node.js and Express.js. Passwords are hashed with bcrypt, sessions are handled with JSON Web Tokens, and role-based access control separates what a student, a faculty member and an administrator may do. The frontend, written in React.js, offers a responsive single-page interface with dashboards for each role. A conflict-detection algorithm guarantees that one slot can never be booked twice, even when two students click at the same moment.

The system supports the complete set of CRUD operations on users, slots and appointments. Functional testing, API testing and basic performance measurements show that the application behaves as specified and responds quickly under normal classroom-scale load. The project demonstrates end-to-end MERN development and provides a foundation that can later be extended with email notifications, calendar integration and analytics.

Keywords: *MERN Stack, Appointment Booking, MongoDB, Express.js, React.js, Node.js, REST API, JWT Authentication, Role-Based Access Control.*

CHAPTER 1 - INTRODUCTION

1.1 Background

Higher education depends on regular, personal interaction between students and teachers. Lectures convey the syllabus, but it is in one-to-one conversations that a student clarifies a concept, receives feedback on a mini project, seeks advice on a career decision or asks for a recommendation. Faculty members, however, have teaching schedules, laboratory sessions, administrative duties and research work that leave only limited windows for such meetings.

When there is no organised mechanism to match student demand with faculty availability, both sides suffer. A student may walk to the department only to find the faculty member is in a class. A faculty member may be interrupted by several students at once while preparing for a lecture. Neither side has a reliable history of past meetings. These are small inefficiencies individually, but over a semester they add up to a significant loss of time and a poorer academic experience.

1.2 Motivation

The idea for this project came from the everyday experience of college life. Appointments are currently arranged through chat groups, handwritten notes on cabin doors or ad-hoc requests after class. Such methods have no confirmation, no reminder, no way to cancel politely and no accountability. A purpose-built system solves these issues and also gives the project team a realistic problem on which to practise full-stack web development.

1.3 Overview of the Proposed System

The Faculty Appointment Booking System is a web application with three kinds of users. A student can register, log in, search for faculty, view free slots, book an appointment with a stated purpose, track the status of each request and cancel when plans change. A faculty member can create and delete slots, view incoming requests, approve or reject them with remarks and mark meetings as completed. An administrator can manage user accounts and see all appointments across the institution.

All data is stored in a MongoDB database in three main collections: users, slots and appointments. The React frontend communicates with the Express backend through JSON REST APIs. Every protected API verifies a JSON Web Token and checks the role of the caller before performing the requested operation.

1.4 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 states the problem in detail. Chapter 3 lists the objectives, scope and limitations. Chapter 4 describes the features and functional requirements. Chapter 5 explains the algorithms and the development methodology. Chapter 6 covers the technologies used and the implementation. Chapter 7 presents testing, results and performance. Chapter 8 concludes the report and outlines future scope, followed by the references.

CHAPTER 2 - PROBLEM STATEMENT

2.1 Problem Definition

The objective of this project is to build a MERN-based faculty appointment booking application in which students can book slots with faculty members. The system must manage student, faculty, date, time and purpose information using MongoDB; React must provide the user interface; and Node.js with Express.js must provide the backend APIs. The system must support the main operations needed for the project, namely adding, viewing, updating and managing the relevant records.

2.2 Limitations of the Existing Approach

In most colleges today, appointments with faculty are arranged informally. Table 2.1 summarises the common problems and the way the proposed system addresses each of them.

Table 2.1: Existing problems and proposed solutions

S.No	Problem in the existing approach	Solution offered by the proposed system
1	Students do not know when a faculty member is free.	Faculty publish slots; students see only free slots.
2	Two students may turn up for the same time.	Server-side conflict detection allows one booking per slot.
3	No confirmation or record of the meeting.	Every appointment is stored with status and purpose.
4	Cancelling or rescheduling needs a phone call or message.	One-click cancellation frees the slot immediately.
5	Faculty cannot prepare because the purpose is unknown.	Purpose is a mandatory field on every booking.
6	No control over who can see or change records.	JWT authentication with role-based access control.
7	Paper registers cannot be searched or analysed.	Searchable MongoDB records with filters by date and status.

2.3 Problem Formulation

Formally, let S be the set of students, F the set of faculty members and T the set of time slots, where each slot t in T belongs to exactly one faculty member f in F and is defined by a date, a start time and an end time. An appointment is a tuple (s, t, p) where s is a student, t is a slot and p is the purpose of the meeting. The system must ensure the following invariants at all times.

1. Every appointment refers to exactly one student and exactly one slot.
2. A slot can be associated with at most one active appointment.
3. Only the owner faculty may modify a slot or decide on a request for that slot.
4. Only authenticated users may read or change data, and only according to their role.

The rest of the report shows how these invariants are enforced by the database schema, the API layer and the user interface.

CHAPTER 3 - OBJECTIVES AND SCOPE

3.1 Objectives

The project has been planned around the following objectives.

- **Demonstrate MERN stack development.** Build a complete application that uses MongoDB, Express.js, React.js and Node.js together, from the database to the browser.
- **Implement CRUD operations using MongoDB.** Create, read, update and delete users, slots and appointments through Mongoose models.
- **Build REST APIs using Node.js and Express.js.** Design resource-oriented endpoints with proper HTTP methods, status codes and validation.
- **Develop the frontend using React.js.** Create a responsive single-page application with reusable components, routing and state management.
- **Implement authentication and role-based access.** Protect every private route using JSON Web Tokens and restrict operations by the roles Student, Faculty and Admin.
- **Display project information and results clearly.** Present slots, appointments, statuses and summaries through clean dashboards, tables and messages.

3.2 Scope of the Project

The scope covers the functions that are essential for a working appointment system inside one institution.

- Registration and login for students, faculty and administrators.
- Faculty profile listing with department and search.
- Creation, listing and deletion of availability slots by faculty.
- Booking of a slot by a student with date, time and purpose captured.
- Approval, rejection, cancellation and completion of appointments.
- Role-specific dashboards showing upcoming and past appointments.
- Administrative management of users and appointments.
- Responsive layout usable on desktop, tablet and mobile browsers.

3.3 Out of Scope and Limitations

To keep the project achievable within the semester, the following items are deliberately left out of the present version and are listed as future work in Chapter 8.

- Automatic email, SMS or push notifications.
- Integration with Google Calendar or the institutional timetable.
- Online video meeting links and payment features.
- Recurring slots generated from a weekly timetable (slots are created individually or in a batch).
- Native mobile applications; the web interface is responsive instead.

3.4 Intended Users

Table 3.1: User roles and their responsibilities

Role	Responsibilities
Student	Registers, searches faculty, books and cancels appointments, tracks status.
Faculty	Publishes and removes slots, approves or rejects requests, completes meetings.
Admin	Manages users, monitors all appointments, removes invalid records.

3.5 Feasibility Study

3.5.1 Technical Feasibility

All components of the MERN stack are open source, well documented and supported by large communities. A developer needs only a laptop with Node.js and a free MongoDB instance, either local or on MongoDB Atlas. The team already has working knowledge of JavaScript, which makes the technology choice technically feasible.

3.5.2 Economic Feasibility

No licensed software is needed. Development tools such as Visual Studio Code, Postman and Git are free, and free tiers of cloud platforms are sufficient for hosting a demonstration. The project therefore has negligible monetary cost.

3.5.3 Operational Feasibility

The interface follows familiar patterns of login, list and form, so users need no training. Faculty spend a minute to add slots, and students spend seconds to book, which makes adoption realistic.

CHAPTER 4 - FEATURES AND FUNCTIONAL REQUIREMENTS

4.1 Key Features

4.1.1 Student Features

- Secure registration and login with email and password.
- Browse the list of faculty members with name and department, and search by name.
- View available slots of a selected faculty member for a chosen date.
- Book a slot by entering the purpose of the meeting.
- View all personal appointments with their current status.
- Cancel an upcoming appointment, which immediately releases the slot.

4.1.2 Faculty Features

- Create availability slots by choosing the date, start time and end time.
- View and delete unbooked slots; booked slots are protected from deletion.
- See incoming appointment requests with student name and purpose.
- Approve or reject a request with optional remarks.
- Mark an approved appointment as completed after the meeting.

4.1.3 Administrator Features

- View, edit and delete any user account.
- View and filter all appointments by status, faculty or date.
- Delete invalid or duplicate records.
- See summary counts of users, slots and appointments on the dashboard.

4.2 Use Case Diagram

Figure 4.1 shows the interaction of the three actors with the main use cases of the system.

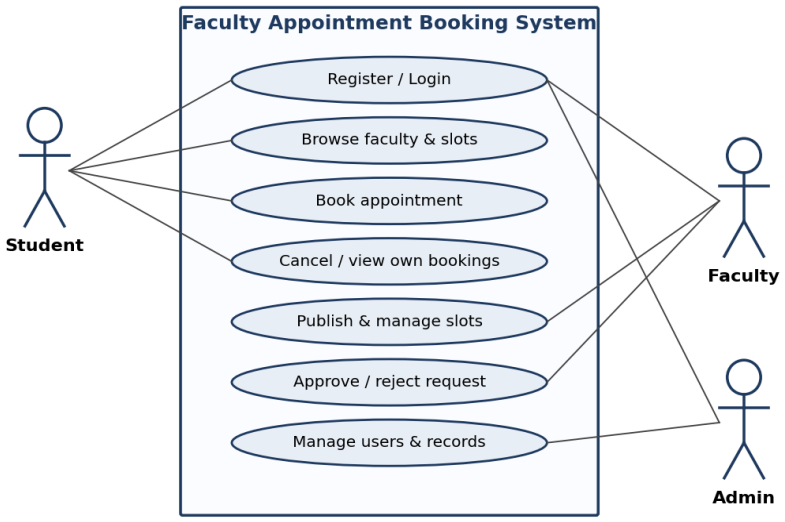


Figure 4.1: Use case diagram of the Faculty Appointment Booking System

4.3 Functional Requirements

The functional requirements describe what the system must do. They are numbered so that they can be traced during testing in Chapter 7.

Table 4.1: Functional requirements

ID	Module	Requirement
FR-01	Authentication	The system shall allow a new user to register with name, email, password, role and department.
FR-02	Authentication	The system shall reject duplicate email addresses and weak passwords.
FR-03	Authentication	The system shall authenticate users and issue a signed JWT with an expiry time.
FR-04	Authorization	The system shall allow an operation only if the role of the caller is permitted for it.
FR-05	Faculty	The system shall list all faculty members and support search by name.
FR-06	Slots	A faculty member shall be able to add a slot with date, start time and end time.
FR-07	Slots	The system shall prevent overlapping slots for the same faculty member.
FR-08	Slots	The system shall allow deletion of a slot only if it is not booked.
FR-09	Booking	A student shall be able to book a free slot by providing the purpose.
FR-10	Booking	The system shall allow only one active appointment per slot.
FR-11	Booking	A student shall be able to cancel an upcoming appointment of his or her own.
FR-12	Decision	A faculty member shall be able to approve or reject a request for his or her own slot.
FR-13	Decision	A faculty member shall be able to mark an approved appointment as completed.

ID	Module	Requirement
FR-14	Dashboard	Each role shall see a dashboard of relevant upcoming and past appointments.
FR-15	Admin	An administrator shall be able to view, update and delete users and appointments.
FR-16	Validation	The system shall validate all inputs on both client and server and return clear error messages.

4.4 Non-Functional Requirements

Table 4.2: Non-functional requirements

Attribute	Requirement
Performance	Typical API calls should respond within 300 ms on a normal network and development server.
Security	Passwords are stored only as bcrypt hashes; tokens are signed; protected APIs reject missing or tampered tokens.
Usability	The interface should be understandable without training and usable on screens from 360 px wide.
Reliability	Booking must be consistent; a slot cannot be double booked even under concurrent requests.
Maintainability	Code is organised in routes, controllers, models and middleware for easy extension.
Portability	The application runs on any operating system that supports Node.js and a modern browser.
Scalability	Stateless APIs allow horizontal scaling; MongoDB indexes keep queries efficient as data grows.

4.5 Appointment Life Cycle

An appointment moves through a small set of states. It is created as Pending when a student books a slot. The faculty member can then Approve or Reject it. A student may Cancel a Pending or Approved appointment before the meeting date. After the meeting, the faculty member marks an Approved appointment as Completed. Rejected and Cancelled appointments free the associated slot so that another student can book it.

Table 4.3: Appointment status transitions

From	To	Done by	Slot after change
(new)	Pending	Student	Booked
Pending	Approved	Faculty	Booked
Pending	Rejected	Faculty	Released
Pending / Approved	Cancelled	Student	Released

From	To	Done by	Slot after change
Approved	Completed	Faculty	Booked (history)

CHAPTER 5 - ALGORITHMS AND METHODOLOGY

5.1 Development Methodology

The project followed an iterative, Agile-inspired approach. Instead of building every feature at once, the work was divided into small increments, each of which produced a working and testable part of the application. This made it easy to correct design mistakes early and to demonstrate progress to the guide at regular intervals.

Table 5.1: Development phases

Phase	Activity	Output
1	Requirement analysis	Problem statement, user roles, functional requirements
2	System design	Architecture, ER diagram, API list, wireframes
3	Backend development	Express server, Mongoose models, authentication, CRUD APIs
4	Frontend development	React pages, components, routing, API integration
5	Integration and testing	Functional tests, API tests with Postman, bug fixing
6	Documentation and demo	Project report, screenshots, presentation

5.2 System Architecture

The application follows a three-tier architecture. The presentation tier is a React single-page application that runs in the browser. The application tier is a Node.js server using Express.js that exposes REST endpoints and contains the business logic. The data tier is a MongoDB database accessed through the Mongoose library. The tiers communicate only through well-defined interfaces, JSON over HTTP between the first two and the Mongoose driver between the last two.

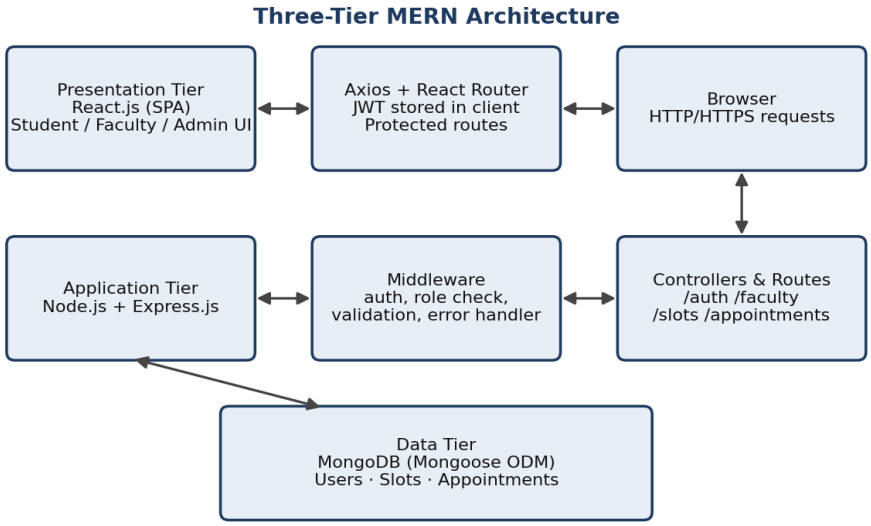


Figure 5.1: Three-tier architecture of the system

When a user performs an action, the React component calls an Axios request to the relevant endpoint. The request first passes through middleware that verifies the token and the role. The

controller then validates the input, performs the database operation and returns a JSON response, which the component uses to update the screen.

5.3 Database Design

MongoDB stores data as flexible JSON-like documents. For this project three collections are enough. Users holds the identity of every person; Slots holds the availability windows published by faculty; Appointments links a student to a slot with a purpose and a status. Relationships are represented with ObjectId references and resolved with the populate method of Mongoose when needed.

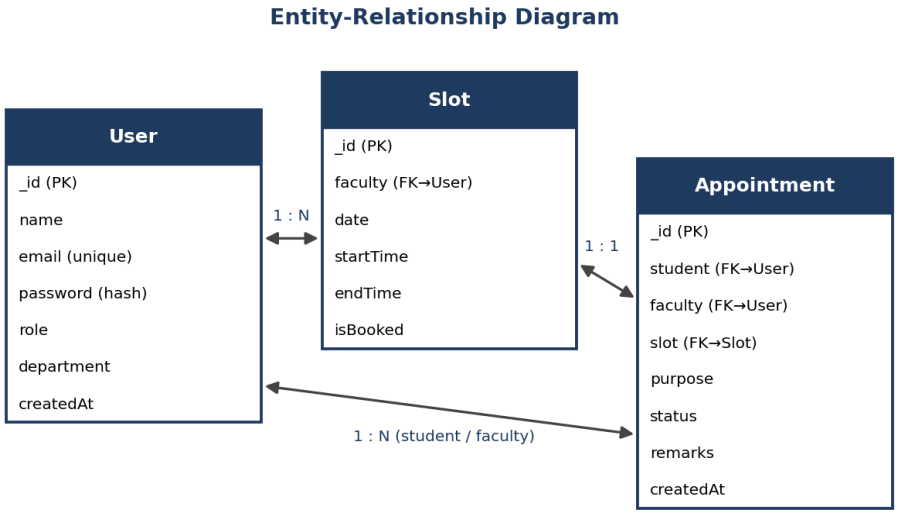


Figure 5.2: Entity-relationship diagram

Table 5.2: Users collection

Field	Type	Description
_id	ObjectId	Unique identifier generated by MongoDB
name	String	Full name of the user (required)
email	String	Unique login identifier, stored in lower case
password	String	bcrypt hash of the password
role	String	One of student, faculty, admin
department	String	Department of faculty or student
createdAt	Date	Automatically added timestamp

Table 5.3: Slots collection

Field	Type	Description
_id	ObjectId	Unique identifier
faculty	ObjectId	Reference to the owning faculty user
date	Date	Calendar date of the slot
startTime	String	Start time in HH:mm format

Field	Type	Description
endTime	String	End time in HH:mm format
isBooked	Boolean	True when an active appointment exists

Table 5.4: Appointments collection

Field	Type	Description
_id	ObjectId	Unique identifier
student	ObjectId	Reference to the student user
faculty	ObjectId	Reference to the faculty user
slot	ObjectId	Reference to the booked slot
purpose	String	Reason for the meeting (required, up to 300 characters)
status	String	pending, approved, rejected, cancelled or completed
remarks	String	Optional note from faculty on the decision
createdAt	Date	Automatically added timestamp

5.4 Algorithms

5.4.1 User Registration with Password Hashing

Passwords are never stored in plain text. During registration the server generates a random salt and computes a bcrypt hash of the password with it. Only the hash is saved. Because bcrypt is intentionally slow and salted, the same password produces different hashes and brute-force attacks become expensive.

```
Algorithm REGISTER(name, email, password, role, department)
1. if any field is missing or password length < 6 then return 400
2. if a user with the same email exists then return 409
3. salt <- bcrypt.genSalt(10)
4. hash <- bcrypt.hash(password, salt)
5. save User(name, email, hash, role, department)
6. return 201 with the created user (without password)
```

5.4.2 Login and Token Generation

On login, the server finds the user by email and compares the supplied password with the stored hash. If they match, it signs a JSON Web Token containing the user id and role, with an expiry time. The client sends this token in the Authorization header of every later request, so the server needs no session storage.

```
Algorithm LOGIN(email, password)
1. user <- find User by email; if none then return 401
2. ok <- bcrypt.compare(password, user.password)
3. if not ok then return 401
4. token <- jwt.sign({ id: user._id, role: user.role }, SECRET, expiresIn 1d)
5. return 200 with token and basic user details
```

5.4.3 Role-Based Access Control

Two middleware functions guard the protected routes. The first, authenticate, extracts and verifies the token and attaches the decoded user to the request. The second, authorize, takes a list of allowed roles and rejects the request with status 403 if the role of the caller is not in the list. For example, only a faculty user can reach the route that creates slots.

```
Algorithm AUTHORIZE(allowedRoles)
1. token <- Authorization header after "Bearer "
2. if token missing then return 401
3. payload <- jwt.verify(token, SECRET); on failure return 401
4. if payload.role not in allowedRoles then return 403
5. attach payload to request; call next()
```

5.4.4 Slot Creation and Overlap Detection

Two slots of the same faculty member on the same date overlap when the start of one is earlier than the end of the other and vice versa. Before saving a new slot, the server queries the existing slots of that faculty member for the date and rejects the new slot if any overlap is found. Times are compared as HH:mm strings, which sort correctly in lexical order when zero padded.

```
Algorithm CREATE_SLOT(facultyId, date, start, end)
1. if start >= end then return 400
2. clash <- exists Slot where faculty = facultyId AND date = date
                    AND start < slot.endTime AND end > slot.startTime
3. if clash then return 409
4. save Slot(facultyId, date, start, end, isBooked = false)
5. return 201
```

5.4.5 Appointment Booking with Conflict Detection

The most important algorithm of the project is booking. A naive implementation first reads the slot, checks that it is free and then writes the appointment. If two students perform this at the same instant, both can read the slot as free and both would book it. To avoid this race condition the server performs the check and the update in a single atomic MongoDB operation, findOneAndUpdate, with the condition isBooked equal to false. Only one request can satisfy the condition; the other receives no document and is told that the slot is already taken.

```
Algorithm BOOK(studentId, slotId, purpose)
1. if purpose is empty then return 400
2. slot <- Slot.findOneAndUpdate({ _id: slotId, isBooked: false },
                                 { isBooked: true }, { new: true })
3. if slot is null then return 409 "Slot not available"
4. appt <- save Appointment(studentId, slot.faculty, slotId,
                            purpose, status = "pending")
5. if save fails then Slot.update({ _id: slotId }, { isBooked: false })
6. return 201 with appt
```

Step 5 acts as a compensating action: if the appointment cannot be saved for any reason, the slot is released again so that the system never leaves a slot locked without an appointment.

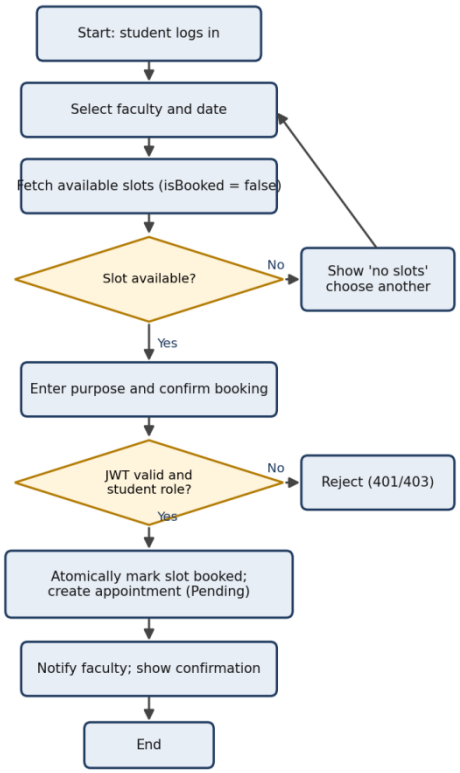


Figure 5.3: Flowchart of the appointment booking process

5.4.6 Cancellation and Decision Handling

```
Algorithm UPDATE_STATUS(userId, role, apptId, newStatus)
1. appt <- find Appointment by apptId; if none then return 404
2. if role = student AND appt.student != userId then return 403
3. if role = faculty AND appt.faculty != userId then return 403
4. if transition(appt.status -> newStatus) not allowed then return 400
5. appt.status <- newStatus; save
6. if newStatus in {rejected, cancelled} then release the slot
7. return 200 with appt
```

5.5 Time and Space Complexity

Each API operation touches a constant number of documents. With indexes on email, on the pair (faculty, date) for slots and on the student and faculty fields for appointments, lookups take logarithmic time in the size of the collection. Overlap detection examines only the slots of one faculty member on one day, which is a very small set. Storage grows linearly with the number of users, slots and appointments.

CHAPTER 6 - TECHNOLOGIES AND IMPLEMENTATION

6.1 Technologies Used

Table 6.1: Technology stack

Layer	Technology	Purpose
Database	MongoDB	Document database storing users, slots and appointments
Backend runtime	Node.js	JavaScript runtime that executes the server
Backend framework	Express.js	Routing, middleware and REST API handling
ODM	Mongoose	Schemas, validation and queries for MongoDB
Frontend	React.js	Component-based user interface
Routing	React Router	Client-side navigation and protected routes
HTTP client	Axios	Calls from React to the REST APIs
Authentication	JWT, bcrypt	Token-based login and password hashing
Styling	CSS / Tailwind CSS	Responsive layout and components
Tools	VS Code, Postman, Git	Editing, API testing and version control

6.1.1 MongoDB

MongoDB is a document-oriented NoSQL database that stores records as BSON documents. Its flexible schema, rich query language and atomic single-document operations suit an application whose entities, such as appointments, are naturally represented as documents with references to other documents.

6.1.2 Express.js

Express is a minimal web framework for Node.js. It provides routing and a middleware pipeline in which every request flows through functions such as body parsing, authentication and error handling before reaching the controller.

6.1.3 React.js

React is a JavaScript library for building user interfaces from reusable components. Its virtual DOM updates only the parts of the page that change, and hooks such as useState and useEffect manage state and data fetching in function components.

6.1.4 Node.js

Node.js executes JavaScript outside the browser using the V8 engine. Its event-driven, non-blocking input and output model allows a single server process to handle many concurrent requests efficiently, which is ideal for an API server that mostly waits for database responses.

6.2 Project Structure

The code is divided into two folders, server and client, so that backend and frontend can be developed, tested and deployed independently.

```
faculty-appointment-system/
|-- server/
|   |-- config/db.js              (MongoDB connection)
|   |-- models/User.js  Slot.js  Appointment.js
|   |-- middleware/auth.js  error.js
|   |-- controllers/authController.js  slotController.js
|   |               appointmentController.js  userController.js
|   |-- routes/auth.js  slots.js  appointments.js  users.js
|   |-- server.js
|   `-- .env                      (PORT, MONGO_URI, JWT_SECRET)
`-- client/
    |-- src/api/axios.js
    |-- src/context/AuthContext.jsx
    |-- src/components/Navbar.jsx  SlotCard.jsx  ProtectedRoute.jsx
    |-- src/pages/Login.jsx  Register.jsx  StudentDashboard.jsx
    |               FacultyDashboard.jsx  AdminDashboard.jsx
    `-- src/App.jsx  main.jsx
```

Figure 6.1: Folder structure of the project

6.3 Backend Implementation

6.3.1 Database Connection and Server Setup

The server reads its settings from environment variables, connects to MongoDB and mounts the route modules under the /api prefix.

```
// server.js
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const mongoose = require('mongoose');

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', require('./routes/auth'));
app.use('/api/slots', require('./routes/slots'));
app.use('/api/appointments', require('./routes/appointments'));
app.use('/api/users', require('./routes/users'));

mongoose.connect(process.env.MONGO_URI)
  .then(() => app.listen(process.env.PORT || 5000,
    () => console.log('Server running')))
  .catch(err => console.error(err));
```

6.3.2 Mongoose Models

```
// models/Appointment.js
const mongoose = require('mongoose');
const { ObjectId } = mongoose.Schema.Types;

const appointmentSchema = new mongoose.Schema({
  student: { type: ObjectId, ref: 'User', required: true },
  faculty: { type: ObjectId, ref: 'User', required: true },
  slot:    { type: ObjectId, ref: 'Slot', required: true },
  purpose: { type: String, required: true, maxlength: 300 },
  status:  { type: String,
             enum: ['pending','approved','rejected','cancelled','completed'],
             default: 'pending' },
  remarks: String
}, { timestamps: true });

module.exports = mongoose.model('Appointment', appointmentSchema);
```

6.3.3 Authentication Middleware

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

exports.authenticate = (req, res, next) => {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ message: 'Login required' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (e) {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
};

exports.authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role)
    ? next()
    : res.status(403).json({ message: 'Access denied' });
```

6.3.4 Booking Controller

The booking controller implements the atomic conflict-detection algorithm described in Section 5.4.5.

```
// controllers/appointmentController.js
exports.book = async (req, res) => {
  const { slotId, purpose } = req.body;
  if (!purpose || !purpose.trim())
    return res.status(400).json({ message: 'Purpose is required' });

  const slot = await Slot.findOneAndUpdate(
    { _id: slotId, isBooked: false },
    { isBooked: true },
    { new: true });
  if (!slot)
    return res.status(409).json({ message: 'Slot not available' });

  try {
    const appt = await Appointment.create({
      student: req.user.id, faculty: slot.faculty,
      slot: slot._id, purpose });
    res.status(201).json(appt);
  } catch (err) {
    await Slot.findByIdAndUpdate(slotId, { isBooked: false });
    res.status(500).json({ message: 'Booking failed' });
  }
};
```

6.3.5 REST API Endpoints

Table 6.2: REST API summary

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Create a new account
POST	/api/auth/login	Public	Authenticate and receive JWT
GET	/api/users/faculty	Logged in	List faculty members
GET	/api/slots?faculty=&date=	Logged in	List slots of a faculty
POST	/api/slots	Faculty	Create a slot

Method	Endpoint	Access	Description
DELETE	/api/slots/:id	Faculty	Delete an unbooked slot
POST	/api/appointments	Student	Book a slot
GET	/api/appointments/my	Student, Faculty	Appointments of the caller
PATCH	/api/appointments/:id/status	Student, Faculty	Change status
GET	/api/appointments	Admin	All appointments
GET	/api/users	Admin	List all users
PUT	/api/users/:id	Admin	Update a user
DELETE	/api/users/:id	Admin	Delete a user

6.4 Frontend Implementation

The frontend is a single-page application. After login, the token and user details are stored in an authentication context so that every component can read them. An Axios instance adds the token to the Authorization header of each request automatically.

6.4.1 Axios Instance and Protected Routes

```
// src/api/axios.js
import axios from 'axios';
const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });
api.interceptors.request.use(cfg => {
  const token = localStorage.getItem('token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});
export default api;

// src/components/ProtectedRoute.jsx
export default function ProtectedRoute({ roles, children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/" />;
  return children;
}
```

6.4.2 Booking Component

The booking form on the student dashboard loads the free slots of the chosen faculty member for the selected date. Selecting a slot and entering the purpose sends a POST request. On success the list is refreshed; on a conflict the user sees the message returned by the server.

```
const handleBook = async (e) => {
  e.preventDefault();
  try {
    await api.post('/appointments', { slotId: selected, purpose });
    setMsg('Appointment requested successfully');
    loadSlots();
  } catch (err) {
    setMsg(err.response?.data?.message || 'Something went wrong');
  }
};
```

6.5 User Interface

The interface uses a clean card and table layout with a navigation bar that changes according to the role of the logged-in user. Forms show validation messages next to the fields, and status badges use colours to make Pending, Approved, Rejected, Cancelled and Completed appointments easy to tell apart. The following figures show the main screens of the app

6.6 Security Measures

- Passwords are salted and hashed with bcrypt; plain-text passwords are never stored or logged.
- JWTs are signed with a secret kept in an environment variable and expire after a fixed period.
- Every protected endpoint passes through the authenticate and authorize middleware.
- Ownership checks ensure a student can change only his or her own appointments and a faculty member only his or her own slots.
- Inputs are validated on the client and on the server; Mongoose schemas reject malformed data.
- Responses never include password hashes, and error messages do not reveal internal details.

CHAPTER 7 - RESULTS, TESTING AND PERFORMANCE

7.1 Testing Strategy

Testing was carried out at three levels. Unit-level checks verified individual functions such as the overlap test and the status-transition rules. API testing with Postman verified each endpoint with valid and invalid inputs and with different roles. Finally, functional testing through the browser verified complete user journeys such as registering, booking and approving an appointment.

7.2 Functional Test Cases

Table 7.1: Functional test cases

ID	Test scenario	Expected result	FR	Status
T01	Register with valid details	Account created; password stored as hash	FR-01	Pass
T02	Register with an existing email	Error 409 shown to the user	FR-02	Pass
T03	Login with correct credentials	Token issued; redirected to dashboard	FR-03	Pass
T04	Login with wrong password	Error 401; no token	FR-03	Pass
T05	Student opens faculty-only route	Error 403 Access denied	FR-04	Pass
T06	Search faculty by name	Matching faculty listed	FR-05	Pass
T07	Faculty creates a valid slot	Slot saved and shown in list	FR-06	Pass
T08	Faculty creates overlapping slot	Error 409 overlap message	FR-07	Pass
T09	Faculty deletes a booked slot	Request rejected	FR-08	Pass
T10	Student books a free slot	Appointment created as pending	FR-09	Pass
T11	Two students book the same slot	Only one succeeds; other gets 409	FR-10	Pass
T12	Student cancels an appointment	Status cancelled; slot released	FR-11	Pass
T13	Faculty approves a request	Status approved	FR-12	Pass
T14	Faculty rejects with remarks	Status rejected; slot released	FR-12	Pass
T15	Faculty marks meeting completed	Status completed	FR-13	Pass
T16	Admin deletes a user	User removed from list	FR-15	Pass
T17	Submit booking form without purpose	Validation message; no request saved	FR-16	Pass

7.3 API Testing

Each endpoint listed in Table 6.2 was exercised using Postman. Requests were sent without a token, with an expired token, with a token of the wrong role and with a valid token. In all cases the server returned the expected HTTP status code and a clear JSON message.

Table 7.2: Sample API test results

Request	Condition	Status code	Result
POST /api/auth/login	Valid credentials	200	Token returned
POST /api/auth/login	Wrong password	401	Invalid credentials
POST /api/slots	No token	401	Login required
POST /api/slots	Student token	403	Access denied
POST /api/appointments	Slot already booked	409	Slot not available
POST /api/appointments	Empty purpose	400	Purpose is required
PATCH /api/appointments/:id/status	Other faculty's appointment	403	Access denied

7.4 Performance Evaluation

Basic performance was measured on a local development machine with the database on the same system, using a tool such as Postman or autocannon to repeat each request and record the average response time. Table 7.3 and Figure 7.2 give the averages for the main endpoints.

Note: the values below are sample figures; replace them with your own measurements before submission.

Table 7.3: Average response time of main endpoints

Endpoint	Average time (ms)	Remarks
POST /api/auth/login	182	Includes bcrypt compare
GET /api/users/faculty	64	Indexed query
GET /api/slots	78	Filtered by faculty and date
POST /api/appointments	141	Atomic update + insert
PATCH /api/appointments/:id/status	96	Single document update

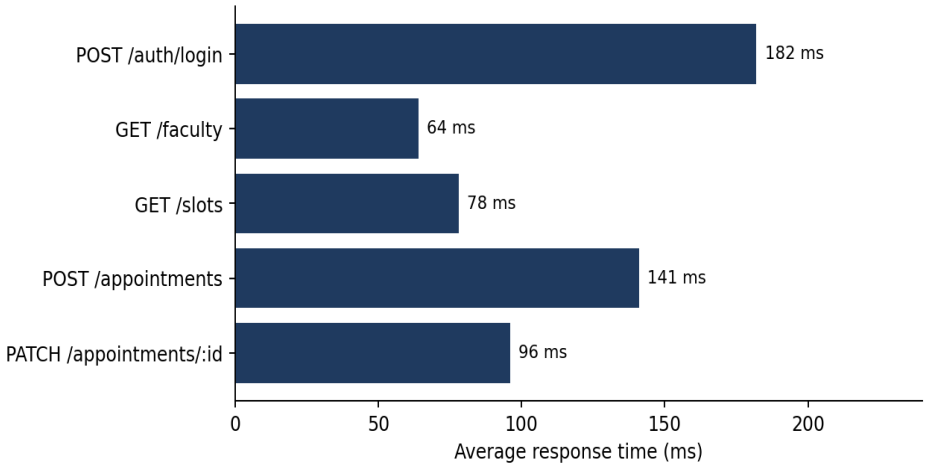


Figure 7.2: Average API response time

The login endpoint is the slowest because bcrypt is deliberately computationally expensive, which is a desirable property for password security. All other endpoints respond in well under 200 ms. These results show that the system is responsive enough for the expected load of a department or college, where only a few requests per second occur at peak times.

7.5 Results

The finished application meets all the objectives listed in Chapter 3. Faculty can publish slots, students can book them with a purpose, faculty can decide on requests and administrators can manage the data. Concurrent booking tests confirmed that the atomic update prevents double booking. Role-based access control correctly blocked every unauthorised operation tried during testing.

Table 7.4: Objectives versus outcome

Objective	Outcome
Demonstrate MERN stack development	Complete application built with MongoDB, Express, React and Node
Implement CRUD using MongoDB	CRUD for users, slots and appointments implemented and tested
Build REST APIs	13 REST endpoints with proper methods and status codes
Develop frontend with React	Responsive SPA with role-specific dashboards
Authentication and role-based access	JWT and three-role authorization enforced on all routes
Display information clearly	Cards, tables, badges and messages show data and results

Figure 7.4: Error message when a slot is already booked

7.6 Observations and Limitations

- The system works best when faculty publish slots in advance; without slots, students cannot book.
- Notifications are shown only inside the application; users must open the dashboard to see changes.
- Time is handled as plain date and time strings, so time-zone conversion is not required for a single campus but would be for multi-campus use.
- Performance was measured on a single machine; load on a deployed server may differ.

CHAPTER 8 - CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project successfully designed and implemented a Faculty Appointment Booking System using the MERN stack. The application replaces informal and error-prone methods of meeting faculty with a structured process in which faculty publish slots, students book them with a clearly stated purpose and every appointment is tracked through its life cycle. MongoDB provides flexible storage for students, faculty, dates, times and purposes; Express.js and Node.js provide secure REST APIs; and React.js provides a responsive and user-friendly interface.

Key technical outcomes include token-based authentication with bcrypt-hashed passwords, role-based access control for three kinds of users, an overlap check that keeps slots consistent and an atomic booking operation that prevents double booking. Testing showed that the system behaves according to its functional requirements and responds quickly under normal use.

Beyond the working software, the project gave the team practical experience of full-stack development: requirement analysis, database modelling, API design, component-based UI development, testing and documentation. The same architecture can be reused for other booking problems such as laboratory slots, counselling sessions or equipment reservation.

8.2 Future Scope

- **Notifications.** Send email, SMS or push notifications when an appointment is booked, approved, rejected or about to start.
- **Calendar integration.** Synchronise appointments with Google Calendar or Outlook and with the college timetable so that slots are suggested automatically.
- **Recurring availability.** Allow faculty to define weekly availability templates and generate slots automatically.
- **Online meetings.** Generate a video meeting link for remote appointments.
- **Analytics.** Provide charts of peak hours, most requested faculty and common purposes to help the administration plan.
- **Feedback and ratings.** Collect short feedback from students after completed meetings.
- **Real-time updates.** Use WebSockets so that slot lists and statuses refresh instantly without reloading.
- **Mobile application.** Build a React Native client reusing the same REST APIs.
- **Enhanced security.** Add refresh tokens, rate limiting, email verification and two-factor authentication.
- **Deployment.** Host the application on a cloud platform with MongoDB Atlas, automated tests and a continuous integration pipeline.

REFERENCES

R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999.

I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.

R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill, 2019.

MongoDB Inc., "MongoDB Manual," MongoDB Documentation. [Online]. Available: https://www.mongodb.com/docs/manual/ (accessed Oct. 2026).

OpenJS Foundation, "Express - Node.js web application framework." [Online]. Available: https://expressjs.com/ (accessed Oct. 2026).

Meta Open Source, "React - The library for web and native user interfaces." [Online]. Available: https://react.dev/ (accessed Oct. 2026).

OpenJS Foundation, "Node.js Documentation." [Online]. Available: https://nodejs.org/en/docs (accessed Oct. 2026).

Mongoose, "Mongoose Documentation." [Online]. Available: https://mongoosejs.com/docs/ (accessed Oct. 2026).

Mozilla Developer Network, "HTTP response status codes." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status (accessed Oct. 2026).

Postman Inc., "Postman Learning Center." [Online].

https://learning.postman.com/(accessed Oct. 2026).